FazeSpike — Product & Creative Strategy Audit

Oct 10, 2026 · @Chimske

FazeSpike's one defensible position is **wavetable-FM**: a full 4x4 FM/PM matrix in which every operator can be any of 381 wavetables, a sampler or a classic shape, with each source-target modulation index individually modulatable. Neither Serum 2, Vital nor Pigments offers that combination. Nothing in the shipping product demonstrates it — the four factory presets use no FM, no wavetable and no arp; a single operator at full level peaks at **-19.3 dBFS**; engaging **Warp 2 silences the synth entirely**; and **Save Preset throws** in both the VST3 and the standalone. Fix the first impression, then build the identity.

What is actually there

The engine is further along than the product. `dsp/synth.js` (1,489 lines) is a real, tested synthesis engine; `npm test` passes end to end and `test/wire.test.mjs` is a genuine UI-to-manifest parity harness. What is missing is everything between a working engine and an instrument someone would choose.

Capability	Verified state	Evidence
4 operators, FM + PM matrix	Works, including per-pair modulated depth. Modulators are built before carriers (topological `opOrder`), so FM is same-block, not one block late	`dsp/synth.js:690-760`

Capability	Verified state	Evidence
Self-FM / self-PM	**Dead in the plugin.** `dsp.main.js:93` does `if (src === k) continue`, so the 8 self knobs never reach the DSP; there is no `op{k}_pm{k}` manifest param either. The engine *can* build it (I rendered it: a tapIn/tapOut pair appears), but through a one-*block* delay (~2.7 ms at 48k/128) — a comb filter, not FM feedback	`plugin/dsp.main.js:93`, `ui/translate.ts:488`
381 wavetables, 8-frame morph	Works, but `wavetables.bin` stores **up to 64 frames per table** and `getBaseWavetableFrame` samples them down to 8 evenly spaced slots. 87% of the morph detail on disk is discarded before it is ever heard	`plugin/PluginProcessor.cpp:1484-1497`
Warp stage 1 (29 modes)	Works. Phase, wave and spectral modes are all ported to C++ and the spectral set is differentially verified bit-exact against the JS reference	`WtSpectralWarp.h`, `test/spectral-warp-diff/`
Warp stage 2	**Non-functional and actively destructive** — see the defects table	`PluginProcessor.cpp:1753`
Modulation: 10 added modules, cable drag, per-knob depth	Works. 41 sources, 82 targets, mirrored across four files and cross-checked by `test:wire`	`src/shared/types.ts`, `test/wire.test.mjs`

Capability	Verified state	Evidence
Mod routes	Hard cap of **24**; route 25 is silently dropped with no warning or counter	`ui/translate.ts:187`
Native arp / sequencer	Substantial and well built: sample-accurate audio-thread clock, 16 patterns, 9 directions, 7 lanes with per-lane length/div/rotation, seeded dice overlay with per-step locks, scale weights, chord layers. **Never heard by a human**	`plugin/ArpEngine.h`
Arp modulation	Works, but all 6 seq targets are built from **voice 0's sources only** — per-voice envelopes and velocity follow whichever note landed in slot 0	`dsp/synth.js:SEQ_TARGETS`
Sampler	Varispeed only: `el.sample` at `rate = base/rootHz`. No crossfade on loop, no time-stretch, no slicing to zones, and no path from a sample to a wavetable. Unverified by ear	`dsp/synth.js:800-830`
FX chain	8 reorderable slots, 8 types, real parametric EQ on native SVF filters. Solid	`dsp/synth.js:1100-1380`
Factory presets	**Four**, all trivial: Init, Sub Bass, Super Saw, Pluck Lead. Not one uses a wavetable, FM/PM, the arp, a warp or a macro	`src/renderer/components/PresetBar.tsx:28-59`

Capability	Verified state	Evidence
Graph rebuild	The *instruction batch* is already cheap — I measured an unchanged re-render at **1 instruction** and a knob change at **2**. The 50-140 ms cost is rebuilding 2,600-4,500 JS node objects in QuickJS to discover nothing changed	measured, `@elemaudio/core` reconcile
Window	Fixed 1272x984, `setResizable(false, false)`. Does not fit a 1366x768 laptop and barely fits 1080p with DAW chrome	`WebViewEditor.cpp:62`
Platform	Windows only; VST3 + Standalone. No AU, no macOS. The UI layer (`choc::ui::WebView`) is portable, so this is build work, not a rewrite	`plugin/CMakeLists.txt:49`

Two things are worth stating plainly. First, the owner's own notes are honest that the entire arp/seq batch, the sampler, glide feel and same-block FM have **never been listen-tested** — and the last time unheard DSP shipped, a base-10 logarithm bug put every note at `f^0.43` and was caught by ear within minutes. Second, `@elemaudio/core@4.0.1` and the vendored `elemaudio/elementary` runtime are both **MIT**, so there is no licensing blocker on the DSP. JUCE 7 Personal is free under 50K USD revenue but requires a splash screen; the 379 third-party Echo Sound Works tables ship with **no written licence in the repo**, which is the real legal exposure.

Critical defects

Every row below was found by reading or running the code in this session, not inferred from the notes. The first four are the reason a new user would uninstall before finding anything good.

Severity	Defect	What the user experiences	Root cause
Blocker	**Warp 2 silences the synth**	Pick any Warp 2 mode on a wavetable operator and the whole instrument goes quiet, and stays quiet through later edits until the runtime self-recovers	`PluginProcessor.cpp:1753` bails with `if (warpMode2 != 0) continue;` before generating any resource, but `dsp/synth.js` still asks `el.table` for a key containing `:w2_<mode>:`. Elementary rejects a node whose resource is missing and **fails the entire instruction batch** — the exact failure mode rule 1 of the overview warns about. Warp 2 has host params, UI, and two mod targets (`opNWarpAmt2`); not one frame of it has ever existed
Blocker	**Save Preset throws**	Name a patch, hit Save, nothing happens. No user preset can ever be saved in the VST3 or the standalone	`ui/shims.ts:17` installs a Proxy that returns a no-op *function* for every `window.api.*` member. `PresetBar.handleSave` therefore takes the Electron branch, `await noop()` gives `undefined` , and `result.presets` is a TypeError. The `localStorage` fallback next to it is unreachable for exactly that reason
Critical	**Gain staging is 25 dB wide**	A single operator at level 1.0 with master maxed peaks at **-19.3 dBFS** (-21.2 at the default master 0.8). Four operators with six notes held peaks at **+4.2 dBFS** — clipped	Measured through `OfflineRenderer` . `HEADROOM_VOICES = 6` pins headroom to `1/sqrt(24)` regardless of how many operators actually sound. There is no output meter, no limiter and no per-patch gain

Severity	Defect	What the user experiences	Root cause
Critical	**Warp Amount is inert for 26 of 29 modes**	Route an LFO to Warp Amount on a spectral or wave mode and nothing moves	`dsp/synth.js:766` computes the modulated `warpAmtNode` but only uses it for modes 3, 4 and 8 (the live phase warps). The resource key is built from the *static* `warpAmtVal`, so every baked mode ignores the route. Worse: for modes 3/4/8 the warp is baked into the frame **and** applied again live — double-warped
Critical	**8 self-FM/PM knobs and 8 mod targets do nothing**	Turn OSC 1's self-PM knob; silence. Route an envelope to `op1Pm1`; silence	`dsp.main.js:93` and `ui/translate.ts:488` both skip `src === k`. The code comments claiming the engine cannot do it are stale — it builds a tapIn/tapOut pair — but the value never arrives
Major	**Modulating WT Pos nearly doubles the graph**	Every knob you turn after routing an LFO to wavetable position costs roughly twice as long to take effect	Measured: 4 WT operators at 12 voices go from **2,601 to 4,565 node creates** because a modulated position abandons the 4-frame static fast path for the full 8-frame weight chain. The most musical thing the synth offers is its worst performance cliff
Major	**Warped frames leak**	Memory climbs through a sound-design session and never comes back	`registeredWavetableWarpKeys` only clears on runtime rebuild or sample load; `pruneSharedResources()` is called from `loadSampleFile` alone. Amount is quantised to whole percent, so one sweep of one warp knob registers 100 key-sets x 8 frames x 8 KB ≈ **6.4 MB**, permanently

Severity	Defect	What the user experiences	Root cause
Major	**Route 25 vanishes**	A complex patch silently loses modulation with no indication which route went	`modToSlots` returns early at `NUM_MOD_SLOTS = 24`. No counter, no warning, no UI state
Major	**FM aliases**	Any non-sine operator taking part in FM/PM switches from BLEP to a naive oscillator. There is no oversampling anywhere in the engine (`grep` for upsample/oversample returns nothing)	`fmOsc` in `dsp/synth.js:95`. Acceptable as a *character* choice, but it is currently an accident, not a decision
Major	**Fixed, non-resizable window**	Will not fit a 1366x768 laptop; on 1080p it fills the screen	`WebViewEditor.cpp:62`. Every competitor is resizable or scalable

One engineering consequence deserves its own line, because it changes the whole performance strategy. I verified that `el.const({key, value})`'s node id is `hashNode('const', props, [])`, which depends **only on the kind string and the key** — never on the value — and that a value change produces exactly `[[3, hash, "value", v], [5]]`. That means C++ can compute the node id for any `ctrl()` key with an eight-line FNV hash and write the property straight into `runtime->applyInstructions`, with **zero QuickJS work**. A knob drag stops being a 50-140 ms full graph rebuild. This is the cheapest large win in the project and nothing in the notes anticipates it.

The identity

FazeSpike is a wavetable-FM instrument: four operators that can each be a wavetable, a sample or a shape, cross-modulating each other at audio rate, driven by a generative sequencer.

That sentence is the only thing in the product that no competitor can copy by adding a feature, because it is a consequence of the architecture rather than a bullet point. The supporting evidence, not the marketing version:

- **A real matrix, not oscillator-to-oscillator FM.** All 16 source-target pairs exist with independent PM and FM depth, and each of the 24 cross pairs has its own mod target (`op1Pm2`, `op1Fm2`, …). An envelope can open *one* modulation index without touching

the other fifteen. Serum 2 feeds FM from any other oscillator but has three oscillators and one depth per destination; Vital's FM is a per-oscillator amount; Pigments' is narrower still. Phase Plant is the only genuine rival here, and it reaches the same place through patch-cord assembly rather than a matrix you can read at a glance.

- **Any operator can be a wavetable.** This is the unusual part. 381 tables usable as *modulators*, not just carriers, means the modulator's own spectrum shapes the sidebands — a formant table modulating a saw is not a sound you can reach in a 3-oscillator synth where FM sources are fixed shapes.
- **Warp feeds FM.** The two warp stages run *before* the operator becomes a modulator, so a spectrally frozen or harmonically rotated table is the modulating waveform. That is a compositional chain (warp → modulate → filter → sequence) nobody else exposes end to end.
- **The sequencer is native and sample-accurate.** `ArpEngine.h` runs on the audio thread before the voice pool, not as a JS timer. Its dice model moves *probability*, never flipping steps, with per-step locks and a seeded PRNG — which means a generated pattern is reproducible and partially protectable. That is the right design and it is already written.

Who it is for

Not "producers" in general, and specifically **not** people shopping on preset count. The audience is the **sound designer who wants movement they did not program**: hybrid-trailer and game-audio people, experimental bass and IDM producers, and the Serum-literate crowd who have run out of things Serum surprises them with. These are people who will open four operators, route a wavetable into another wavetable's phase, roll the dice, and keep the accident.

The practical consequence: that audience judges a synth in ninety seconds by playing presets. Today FazeSpike fails that test completely — four presets, none of which uses anything described above. **The identity exists in the code and nowhere in the product.**

The promise

> *Sounds that keep moving. Four wavetables modulating each other, a sequencer that rolls its own dice, and a lock on every happy accident.*

Where FazeSpike cannot win

It will not beat Serum 2 on engine breadth (granular, spectral, multisample), Vital on price (free), or Pigments on factory content and polish. Competing on feature count is a loss by arithmetic: those teams ship more per quarter than one developer can. The only winnable game is **depth in one direction plus a workflow nobody else has**.

Signature workflows

Three workflows that exploit what is already built rather than adding systems. Each one is reachable from the current architecture; none requires a rewrite.

1. The FM Matrix as an instrument, not a form

Today the modulation cluster is eight knobs per operator card — four PM, four FM — spread across four cards. Sixteen pairs, thirty-two knobs, no overview. The *structure* of an FM patch, the thing that determines whether it is a bell or a bass, is invisible.

Replace it with a single 4x4 grid. Rows are targets, columns are sources, each cell is a two-handle control (PM left, FM right) tinted with the source's colour. The diagonal is self-modulation. Drag a cable from an LFO onto a cell to modulate that one index; drag across cells to paint a diagonal or a chain.

What the musician experiences: they can *see* a Yamaha algorithm. A 2-op stack is one lit cell; a 4-op chain is a lit diagonal; feedback is a lit diagonal cell. Changing algorithm becomes a gesture, not a hunt through four cards. Add six or eight preset algorithm stamps (stack, chain, parallel, double-stack, ring) as one-click fills and an FM novice gets a usable starting structure in a second.

Why it is better: no wavetable synth shows FM topology as a readable object. Serum has no matrix to show. Phase Plant shows routing but as cables, which is powerful and slow. This is the single highest-leverage UI change available, and it costs no DSP.

2. Sample → wavetable, in one gesture

The sampler is currently a dead end: a sample plays varispeed and that is all it does. But `dsp/wavetables.js` already contains `analyzeFrame` and `synthesizeFrame` (a 128-harmonic DFT), `UserWavetables.h` already registers user tables, and `WtSpectralWarp.h` already ports every spectral transform to C++. Everything needed for resynthesis is in the repo, unconnected.

The workflow: load or record a sample, drag-select a region (the `SamplerBlock` highlight already does this), and hit **→ WT**. FazeSpike slices that region into 64 single-cycle frames at the detected pitch, writes them as a user wavetable, and switches the operator to wavetable mode with the new table selected and WT Pos at 0. The morph across the table *is* the evolution of the original sound, now scannable, warpable, and — critically — **usable as an FM modulator**.

What the musician experiences: they sing one vowel, drag it, press one button, and have a formant wavetable they can sweep with an LFO and modulate a bass with. Then they resample *that* and do it again. The loop closes.

Why it is better: Serum 2 and Vital both import audio to wavetables. Neither lets you then use the result as a modulator in a 4x4 matrix, and neither puts resynthesis one button from the sampler region you just trimmed. The differentiation is the *loop*, not the resynthesis.

3. Lock the accident

The arp's dice already have per-step locks and a seeded PRNG. That mechanism is more valuable than the arp. **Extend locking to the whole patch.**

A **Mutate** button re-rolls the patch within constraints: wavetable choice, warp mode and amount, FM/PM depths, filter, envelope times. Every control has a padlock (reuse the existing lane-lock affordance and visual language). Lock the FM matrix and mutate only timbre; lock the wavetables and mutate only modulation; lock everything but Warp 2 and crawl through 29 modes. A **Mutate Amount** knob scales how far each roll travels, mirroring the dice's Random Amount, and the same seeded PRNG makes any result reproducible and shareable as a number.

What the musician experiences: a patch they half-like becomes twenty variations in twenty seconds, each one a real patch rather than noise, because the locks hold the parts they chose.

Why it is better: Vital has randomisation, Pigments has dice on the sequencer, and both are blunt. A **lockable, scalable, seeded mutation over the whole patch** — using the exact mental model already established on the arp lanes — is a workflow, not a button. It is also the cheapest way to generate the factory library this product does not have.

Distinctive concepts

Four concepts, deliberately few. Each is scored in the next section. I have left out the obvious expansions (granular, more FX, more engines) because they are the anti-roadmap.

A. Native wavetable oscillator node (`wtosc`)

The idea. Move the wavetable oscillator out of the Elementary graph and into a C++ custom node, exactly as `voicectl` and `ctlout` already are — both are about 40 lines.

The experience. Nothing visible, and that is the point: the morph becomes smooth instead of stepped (64+ frames instead of 8), modulated WT Pos stops costing a performance cliff, Warp Amount becomes a live continuous control for *every* mode instead of 3 of 29, and warped frames stop leaking.

The differentiation. None by itself. It is the precondition for everything else. Every other wavetable concept in this document is gated on it.

The implementation. `plugin/WtOsc.h` on the pattern of `CtlOut.h`: properties `table`, `frames`, `warp1/2 mode+amount+param2`; inputs `[phase, position, warpAmt1, warpAmt2]`; reads frames directly from the parsed `wavetables.bin` (`WavetableBin.h` already gives random access to any frame) and interpolates in the audio callback. `dsp/synth.js`'s `wavetableOsc()` collapses from ~70 nodes per operator per voice to one `createNode('wtosc', ...)`. Phase-domain warps move into the node; spectral warps stay baked but become cacheable per *table*, not per quantised amount.

The MVP. One operator, phase-domain warps only, spectral modes still routed through the existing baked path. Prove it against `engine.test.mjs` by rendering the same patch through both paths and comparing spectra.

The risks. Spectral warps genuinely need precomputation — a 2048-point DFT per frame cannot run per block — so the node needs a dual path and the audio thread must never allocate. Mitigation: keep `ensureWavetableWarpResources` for spectral modes, generate on the message thread into a double-buffered frame set, and swap atomically. Second risk: `dsp/synth.js` is shared with the browser preview, so the `wtosc` path needs the same `nativeVoiceCtl()`-style flag the voice control already uses.

The commercial value. Indirect but large. It converts "the wavetable morph sounds steppy" into "the morph sounds like Serum", and it is what makes a demo video of a modulated 4-operator patch run without a stutter.

B. Resynthesis loop (Sample → Wavetable → FM source)

The idea. Workflow 2 above, shipped as a single button on the sampler card.

The experience. Record eight bars of your own voice, a guitar, or FazeSpike's own output; drag a region; press **→ WT**; the operator becomes a wavetable of that sound; route it into OSC 2's phase. Press **→ WT** again on the result and you are two generations deep into a sound that is unmistakably yours.

The differentiation. Audio-to-wavetable exists in Serum 2 and Vital. The *loop* — resynthesise, modulate, resample, resynthesise — with the result immediately available as an FM modulator in a 4x4 matrix, does not. It also gives FazeSpike a story Vital's text-to-wavetable cannot: everything happens offline, on your machine, with your own material.

The implementation. Reuse `analyzeFrame` / `synthesizeFrame` in `dsp/wavetables.js`, ported to C++ beside `WtSpectralWarp.h`'s DFT. Pitch detection by autocorrelation over the selected region. Write to `Documents/Faze/Wavetables` as a standard 2048-sample-per-frame WAV so `UserWavetables.h` picks it up with no new format. The one real

decision is **preset portability**: user wavetable ids are positional today, so a preset referencing a resynthesised table breaks on another machine. Fix it by storing name plus content hash and embedding the table in the preset when it is user-generated.

The MVP. Fixed 32 frames, pitch from the sampler's existing root-note picker rather than detection, no embedding — a local-only feature — behind a clear label. Enough to demo and to judge whether the sound justifies the rest.

The risks. Resynthesis quality is the whole feature; a 128-harmonic DFT at one frame per slice will sound dull on anything percussive, and the only way to know is to listen. Pitch detection on inharmonic material will be wrong often enough to need a manual override. Preset portability is a saved-state compatibility question that must be decided before the first public preset ships, not after.

The commercial value. High and easy to demonstrate: "sing into your synth" is a thirty-second video. It attracts the sample-based and vocal-chop audience that currently has no reason to look at an FM synth.

C. Lockable patch mutation (seeded)

The idea. Workflow 3 above: whole-patch Mutate with per-control locks, a Mutate Amount, and a seed.

The experience. Find a patch that is 70% right. Lock the FM matrix and the envelope. Hold Mutate. Twenty variations go past; one is better than anything you would have dialled. Its seed is in the title bar, so you can get back to it, and the undo history (already built, already coalesced at 500 ms) catches everything else.

The differentiation. Randomise buttons are everywhere and nobody uses them twice, because they destroy the parts you liked. Locks are what make mutation usable, and FazeSpike has already shipped the lock interaction on the arp lanes — so this is a consistent idea extended, not a new one bolted on.

The implementation. Entirely in `src/shared/types.ts` and the UI; **no DSP work at all**. A `locks` set on `SynthSettings`, a mutation function per parameter kind (continuous: gaussian scaled by amount; discrete: weighted neighbour; wavetable id: within the same category), the existing `onChange` path, and the existing undo stack. The seeded PRNG can be lifted from `ArpEngine.h`.

The MVP. Mutate with no locks, amount fixed at 25%, over wavetable id, warp mode/amount, FM/PM depths and filter cutoff only. One afternoon. Judge by whether the rolls are usable.

The risks. Low technically; the whole risk is taste. A mutation range that is too wide produces garbage and the feature dies on first use. This needs tuning by ear against real patches, per parameter, and that tuning *is* the work. Second risk: every mutation is a full

state swap, so it triggers a graph rebuild — at 50-140 ms, holding Mutate will stutter until concept A or the const fast path lands.

The commercial value. Strong. It is the feature that writes the factory library, and "lock what you like, roll the rest" demos in ten seconds.

D. Patch-wide evolution: the sequencer drives timbre

The idea. The arp already exposes six modulation *targets* (its own rate, swing, gate, transpose, velocity, randomness). Invert it: let the sequencer's lanes be modulation **sources** for the synth, so a step pattern drives wavetable position, warp amount and FM index per step.

The experience. Draw a 16-step pattern in the pitch lane. Assign a second lane to OSC 1's WT Pos and a third to the OSC 2→OSC 1 FM index. Now each step has its own timbre, locked to the grid, and the dice can re-roll the *timbre* pattern while the notes stay put. Roll the dice on the warp lane and every eighth note is a different spectral transform.

The differentiation. This is the genuinely new idea in this document. Pigments' sequencer modulates its own parameters; Serum 2's arp is a note generator. A sequencer whose lanes are first-class modulation sources into an FM matrix, with per-step dice and locks, is a composition tool, not an arpeggiator. It is also what the title promise — *sounds that keep moving* — actually means.

The implementation. The hard part is already done, in the right direction: `ctlout` proves the graph can read native values, and `voicectl` proves the audio thread can push per-voice state into the graph. Add `seqlane` as a third custom node — `createNode('seqlane', { lane })` reading an atomic bank the `ArpEngine` writes at each step — and append it to `MOD_SOURCES` (append-only, so existing automation is safe). The UI already has lanes, dice, locks and rotation.

The MVP. Two spare lanes (the arp mode ignores the gate and pitch lanes already) exposed as sources `seqLaneA` / `seqLaneB`, routable by the existing cable drag to any target. No new UI beyond two source dots.

The risks. Modulation reaches the graph one block late, same as `ctlout` — fine for per-step timbre, audible if someone expects sample-accurate. A per-step change to WT Pos will *not* retrigger the wavetable fast path, so this depends on concept A to be smooth. Biggest risk is conceptual load: the arp/seq UI is already dense and this adds a second meaning to every lane; it needs a clear mode switch rather than a hidden one.

The commercial value. This is the one that earns coverage. It is also the hardest to explain in a still image, which is a genuine marketing cost — it demands a video.

Competitive landscape

The honest summary: on every axis FazeSpike currently competes on, it loses. The table is what the competitors actually ship, from pages opened for this audit.

	FazeSpike (today)	Serum 2	Vital	Pigments 7	Phase Plant
Price	unpriced	$249	free / $25 tier	~$199 ($99 intro)	~$199 + Snapin packs
Oscillators	4, each WT / sample / shape	3, each WT / sample / multisample / granular / spectral	3 WT + sample	6 engines (VA, WT, granular, harmonic, modal, sample)	unlimited generators
FM topology	**full 4x4 PM+FM matrix, per-pair modulated index**	FM/PD/AM/ring from any other oscillator	per-oscillator FM amount	limited	audio-rate mod between any generators, via cables
Warp	2 stages, 29 modes, serial	**2 warp engines, 32 warp/filter/distortion options**	15 spectral morph + 6 wave morph	per-engine	Snapin chain
Resynthesis	none	audio import	audio import, pitch-splice, vocode, text-to-wavetable (online)	audio import	sample import
Sequencer	native, dice, polymetry, 7 lanes, locks	arp + sequencer	arp	"generative sequencing"	modulators, no step sequencer of this kind
Factory presets	**4**	626+, 288 wavetables	75 (free tier), 25 wavetables	150 new in v7 alone	large library
Platforms	Windows, VST3 + Standalone	Win/mac (+ Linux beta)	Win/mac/Linux	Win/mac	Win/mac
UI	fixed 1272x984	resizable	resizable	resizable	resizable

What this means

Two warp stages is not a differentiator. Serum 2 has had two warp engines with 32 options for a year. Claiming novelty here would be exactly the mistake of calling a feature unique because it has a different name. FazeSpike's warp advantage, if it has one, is that warp output feeds the FM matrix — a *routing* claim, not a count claim.

The FM matrix is the only real gap. Serum 2's three oscillators and single depth per destination cannot express a 4-operator chain with independently modulated indices. Phase Plant can, but by assembling it from cables, which costs the at-a-glance readability a matrix gives. A searchable claim that nothing in the market answers: *"a wavetable synth where any of 381 tables can modulate any other table's phase, with each of the 16 indices separately modulatable."*

Preset count is an existential problem, not a polish item. 4 against 626. No reviewer, no YouTuber and no buyer will get past that number, and it is also the only way most people will ever discover the FM matrix. This is why concept C (lockable mutation) matters commercially more than its technical ambition suggests — it is the preset factory.

Vital being free sets the floor. Any price FazeSpike charges must be justified against something Vital cannot do. "More wavetables" is not that thing; Vital has text-to-wavetable and a converter suite. "Four wavetables modulating each other, locked mutation, and a sequencer that modulates timbre" is.

Underserved workflows I could not find anyone serving:

1. A **readable FM algorithm grid** in a modern wavetable synth. Everyone either hides FM behind a per-oscillator knob or exposes it as cables.
2. **Locked, scaled, seeded whole-patch mutation.** Randomise buttons are universal; locks on them are not.
3. **Sequencer lanes as timbre modulation sources.** Pigments modulates its sequencer; nobody sequences the FM index per step with dice and locks on the pattern.
4. **An offline, local resynthesis loop.** Vital's most talked-about generator is a web service. "Your material, your machine, no account" is a position.

The risk of competing on features

One developer against Arturia and Xfer loses a feature race by arithmetic. The specific trap visible in the current notes is a long tail of parity work — more FX types, more warp modes, more LFO shapes, more arp patterns — each individually cheap and none of which changes why anyone would choose this synth. The 29 warp modes are already a case in point: 26 of them cannot be modulated, so the count is larger than the usable feature.

Priorities

Scored 1-5 on each axis. **Feasibility 5 = easiest. Risk 5 = highest risk.** The four bands are ordered by what should be done first, not by total score — a reliability fix with low distinctiveness still comes before an ambitious feature.

Band 1 — Immediate fixes (days; do these before anything else)

Work	Impact	Distinct	Commercial	Feasibility	Risk	Reasoning
Warp 2 silence	5	1	4	5	1	A documented, host-automatable feature that mutes the instrument. Either generate the stage-2 resource or refuse to emit a warp-2 key — both are small. Commercial 4 because total silence is the kind of bug that ends a trial
Save Preset throws	5	1	5	5	1	Nobody can keep a sound. Two lines: give the Proxy an explicit `undefined` for preset verbs, or implement them against the native state chunk. Commercial 5 — a synth you cannot save from is not a product
Gain staging	5	2	5	4	2	-19.3 to +4.2 dBFS with no meter. Fixing it means choosing a reference level and compensating per sounding operator, which **changes the loudness of every existing patch** — hence risk 2, not 1. Do it now, while there are only four presets to re-level
Mod-route cap warning	3	1	2	5	1	Silently losing route 25 is a trust bug. A counter and a refusal are an afternoon
Resizable / scaled window	4	1	4	3	2	Blocks adoption on laptops outright. Feasibility 3: the layout is pixel-pinned at 1272x984 in several places and the notes record a frozen height budget, so a CSS zoom wrapper is the realistic first step, not true reflow

Band 2 — Low-cost, high-leverage (1-2 weeks)

Work	Impact	Distinct	Commercial	Feasibility	Risk	Reasoning
Const fast path (native prop write)	5	3	4	4	2	Verified: const node id = hash of kind + key only; a value write is one instruction. Turns every knob drag from 50-140 ms into nothing. Feasibility 4 because it needs an FNV hash mirrored in C++ plus a map from param id to `ctrl()` key, and a correctness test that the hash matches `@elemaudio/core`. Distinct 3 — invisible, but it is what makes concepts C and D usable
20-30 factory presets using the real engine	5	3	5	4	1	The single highest commercial return in the document. Must be made **by ear**, must use wavetable FM, warps, the arp and macros, and must be the vehicle that finds the remaining sonic bugs
Listen-test the arp/seq, sampler, glide	5	1	4	4	3	A large, well-built, entirely unheard subsystem. Risk 3 because the probable outcome is discovering real bugs — the log-base-10 glide bug is precedent
Self-FM/PM: wire it or remove it	3	3	2	4	2	8 knobs and 8 mod targets that do nothing is worse than absent. If wired, the one-block feedback delay must be fixed or it will sound wrong; if removed, the mod-target indices must stay reserved (append-only)
Fix Warp Amount modulation + double-warp	4	3	3	3	2	26 of 29 modes ignore their mod route, and 3 apply the warp twice. Proper fix is gated on concept A; an interim fix is to stop baking modes 3/4/8 and mark the rest as static in the UI

Band 3 — Product investments (1-2 months each)

Work	Impact	Distinct	Commercial	Feasibility	Risk	Reasoning
A. Native `wtosc` node	5	3	4	3	3	Fixes the 8-frame morph, the WT Pos performance cliff, the frame leak and live warp in one change. Feasibility 3: precedented twice in this repo, but the spectral dual path and real-time safety are genuine work. Risk 3 — it touches the single most-used audio path
C. Lockable mutation	5	5	5	4	2	No DSP work, high distinctiveness, and it generates the preset library. The risk is entirely taste: mutation ranges that produce garbage kill it on first use. Should ship after the const fast path so holding Mutate does not stutter
FM matrix grid UI	5	4	4	4	2	Makes the identity visible and learnable. Pure UI; the state model already holds everything. Risk 2: it replaces a layout the owner has iterated on heavily, so it needs mockup sign-off before building
B. Resynthesis loop	5	4	5	3	4	The best demo in the document and the widest new audience. Risk 4: quality is unknown until heard, pitch detection will be wrong on inharmonic material, and preset portability for user tables is an unresolved saved-state question that must be settled *before* public presets ship

Band 4 — Long-term bets

Work	Impact	Distinct	Commercial	Feasibility	Risk	Reasoning
D. Sequencer lanes as mod sources	5	5	4	3	3	The most original idea here and the clearest expression of the promise. Infrastructure exists (`ctlout`, `voicectl`, lanes, dice, locks); the cost is conceptual load on an already dense UI, and it only sounds right once `wtosc` lands
macOS + AU	4	1	5	2	3	Roughly doubles the addressable market. `choc::ui::WebView` is cross-platform and Elementary is MIT, so this is build-system, signing and notarisation work rather than a rewrite — but it is weeks of it, plus a second platform to maintain forever
Oversampled FM path	3	2	2	2	3	Non-sine FM aliases today because `fmOsc` is naive and nothing oversamples. Fix properly or **decide it is the character** and say so. Low priority either way; dirt is acceptable in this genre, undocumented dirt is not
Per-voice arp modulation	2	2	1	2	3	All six seq targets read voice 0's sources. Correct fix needs per-voice `ctlout` banks. Narrow benefit for real architectural cost

Dependencies that constrain the order

- **Gain staging before presets.** Re-levelling after a library exists means re-levelling the library.
- **Const fast path before mutation.** Each mutation is a full state swap; at 50-140 ms, holding Mutate stutters.
- **`wtosc` before sequenced timbre.** Per-step WT Pos changes abandon the static fast path and double the graph.
- **User-wavetable preset portability before any public preset uses a resynthesised table.** Ids are positional today; a preset that references one breaks on another machine.

- **Mod source/target lists stay append-only, always.** Four files mirror them and `test:wire` enforces it. Every concept here appends; none reorders.
- **`test:wire` is currently red** (15 Audio Follower default mismatches: `af_rel` 50 vs 120, `af_gain` 15 vs 12, `af_src` 1 vs 0). A red harness cannot protect the work above — fix it in band 1.

Roadmap

Product identity

> **FazeSpike — four wavetables modulating each other, and a sequencer that rolls its own dice.**

For the sound designer who wants movement they did not program: hybrid-trailer and game-audio people, experimental bass and IDM producers, and Serum-literate producers who have run out of surprises. Not for preset-browsers, and not a Serum replacement.

The three highest-priority investments

1. Make the first ninety seconds work. Warp 2 silence, Save Preset, gain staging, the red `test:wire`, and 20-30 factory presets built by ear out of the real engine. Justification: a new user currently meets a synth that is 20 dB too quiet, cannot save, goes silent if they touch Warp 2, and ships four presets that use none of its distinctive features. Every one of those is verified in this session. No amount of later work recovers a trial that ends in the first minute, and the presets are also the only mechanism that will surface the sonic bugs in the arp, sampler and glide that nobody has heard yet.

2. The const fast path. Mirror `@elemaudio/core`'s const-node hash in C++ and write knob values straight into `runtime->applyInstructions`, bypassing QuickJS. Justification: I verified the hash depends only on kind and key, and that a value change is exactly one `SET_PROPERTY` instruction plus a commit. Today a knob drag rebuilds 2,600-4,500 JS node objects to discover nothing structural changed, at 50-140 ms on the message thread. This single change removes the project's defining performance constraint, and it unblocks mutation, sequenced timbre and modulated wavetable position — all three of which are currently throttled by it.

3. Lockable patch mutation. Whole-patch Mutate with per-control locks, a Mutate Amount and a seed, reusing the lock language already shipped on the arp lanes. Justification: it needs no DSP work, it is the highest-distinctiveness item in the scoring,

and it is the only realistic way a one-developer product produces a competitive preset library. It also directly serves the stated promise — sounds that keep moving — at the patch level rather than the note level.

The signature feature

The sequencer modulates the timbre, not just the notes (concept D). Sequencer lanes become first-class modulation sources into the FM matrix, wavetable position and warp amount, with the existing per-step dice and locks operating on the *timbre* pattern.

This is the one feature that is genuinely not available elsewhere and that no competitor reaches by adding a module — it needs a native sequencer already inside the voice path and an FM matrix worth sequencing, and FazeSpike has both. It is also the feature that makes every other investment pay: the FM matrix becomes something you compose with, the resynthesised wavetables become something that changes per step, and mutation becomes something that writes patterns as well as patches.

Sonic identity

Recognition comes from a sound people can name, not a UI they can recognise. Four concrete commitments:

1. **One signature texture, in every demo.** Wavetable-into-wavetable PM with a slow warp sweep and a dice-rolled sequence underneath — metallic, breathing, never static. Put it in the Init-adjacent presets so it is the first thing anyone hears.
2. **Name the preset categories after behaviour, not instrument class.** Not Bass/Lead/Pad but **Breathing, Clangorous, Collapsing, Rolling, Vocal-ish**. The categories become the claim: this synth makes sounds that *do* something.
3. **Decide the dirt.** Non-sine FM aliases because `fmOsc` is naive. Either oversample it or declare the grit part of the character and build presets that use it deliberately. An undecided artefact reads as a bug; a decided one reads as a sound.
4. **Demos are 20-second loops, not feature tours.** One patch, one gesture (a macro, a dice roll, a Mutate hold), audible change. The things worth filming are: Mutate with half the patch locked; a vocal sample becoming a wavetable and then an FM modulator; one lane re-rolled so every step changes timbre.

The anti-roadmap

Things to deliberately not build, and why:

- **A granular engine.** Serum 2 and Pigments both have one and both are better resourced. It would consume the whole roadmap and arrive as the fifth-best option.

- **More FX types.** Eight slots and eight types is already sufficient and not why anyone would buy this. The FX chain is table stakes, not a differentiator.
- **More warp modes.** 29 exist and 26 cannot be modulated. Make the existing ones work before adding a thirtieth.
- **More arp patterns past 18.** The patterns are not the feature; the dice, the locks and the lanes are.
- **A cloud preset browser, accounts, or any online dependency.** "Your material, your machine, no account" is a position worth keeping, and it is free to keep.
- **AI anything.** Nothing in this document needs a model. Seeded, locked mutation is a better creative tool than generation because it is reproducible and controllable, and it does not need a server, a subscription or an explanation.
- **A modular patch-cord view.** Phase Plant owns that and does it well. The matrix is the opposite bet and the better one for this product.
- **More voices past 12.** Idle voices already cost full CPU in the graph; adding voices makes the real problem worse.
- **A second preset format or a migration layer.** The state chunk is a flat map and it works. Keep it, keep it append-only.

Phased plan

Phase	Contents	Gate (must all pass to move on)
P0 — Trustworthy (1-2 wks)	Warp 2 silence; Save Preset; mod-route cap warning; `test:wire` green; gain staging with an output meter	`npm test` and `npm run test:wire` both green. A new regression test proves a warp-2 patch renders a non-silent batch. Save/reload a user preset in Reaper and in the standalone. Measured: a 1-op patch at default master sits within 6 dB of a 4-op patch with 6 notes held, and nothing clips
P1 — Responsive (2-3 wks)	Const fast path; CSS-zoom resizable window; listen-test and fix the arp/seq, sampler, glide	A knob drag on the heaviest patch logs **zero** `dispatchStateChange` reconciles. A C++ unit test asserts the const hash matches `@elemaudio/core` for 20 real `ctrl()` keys. Owner has played the arp at 8-20 Hz through every direction and signed off by ear. UI usable at 1366x768

Phase	Contents	Gate (must all pass to move on)
P2 — Recognisable (3-4 wks)	20-30 factory presets by ear; behaviour-named categories; FM matrix grid UI (mockup approved first); self-FM decision	Every preset uses at least two of {wavetable FM, warp, arp, macros}. Three producers who have not seen the synth can find a sound they want within 2 minutes of browsing. Matrix grid reproduces any existing patch's routing identically
P3 — Distinctive (4-6 wks)	Native `wtosc`; lockable mutation	`wtosc` and the old path render spectrally equivalent output for 10 reference patches (same test shape as `spectral-warp-diff`). 4 WT operators with modulated position at 12 voices costs no more graph than static position. Mutate held for 10 s produces 20 patches of which the owner keeps at least 3
P4 — Signature (6-8 wks)	Sequencer lanes as mod sources; resynthesis loop; then macOS/AU	Two lanes routable to any target via the existing cable drag, verified by ear at 1/16 with dice. A sung vowel becomes a wavetable and then an FM modulator in under 30 s, and the result is usable in a preset. Preset portability for user tables decided and implemented before any ships

Validation is by ear at every gate, not by test suite alone — that is the lesson the glide bug already taught this project. Each phase should end with the owner making one finished 8-bar musical idea using only that phase's work, and saying whether the synth got better to use.

The next five engineering tasks

In order. Each is small enough for one session and has a test that must pass before it counts as done.

1. Warp 2: stop the silence. `plugin/PluginProcessor.cpp:1753`, `dsp/wavetables.js`, `dsp/synth.js`.

Remove the `if (warpMode2 != 0) continue;` early-out and generate stage-2 frames by feeding stage 1's output through the same three dispatch paths (`wtIsPhaseDomainWarp` / `wtIsWaveDomainWarp` / `wtIsSpectralWarp`), matching `generateWavetableFrames`'s two-

pass order in the JS reference. If that is more than one session, ship the safe half first: make `wavetableResourceKey` in `dsp/synth.js` ignore stage 2 entirely so the graph can only ever ask for a key C++ will generate.

Expected: selecting any Warp 2 mode changes the sound and never silences it. *Test:* a new case in `engine.test.mjs` that renders a patch with `warpMode2 = 16, warpAmount2 = 0.5` and asserts non-zero RMS; plus `[wtdiag] applyInstructions ok` in `%TEMP%/fazespike-ui.log` after sweeping every Warp 2 mode in the standalone. Also confirm `warpMode2 != 0` with `warpAmount2 == 0` is safe — that case silences today too.

2. Save Preset: make it work in the plugin. `spike/elementary/ui/shims.ts:17`, `src/renderer/components/PresetBar.tsx:266`.

The Proxy returns a no-op *function* for every member, so `PresetBar` takes the Electron branch and dereferences `undefined`. Either return `undefined` for `savePreset` / `listPresets` / `deletePreset` so the existing `localStorage` fallback runs, or — better — add real bridge verbs that write a `.fazepreset` JSON next to the user wavetable folder, so presets survive a reinstall and can be shared.

Expected: naming a patch and pressing Save keeps it, and it is in the browser after a restart. *Test:* save, close, reopen the standalone, load it, and diff the recalled `SynthSettings` against what was saved; repeat in a DAW to confirm it is independent of the host's own state chunk.

3. Gain staging and an output meter. `dsp/synth.js:1430` (`HEADROOM_VOICES`, `headroom`), `plugin/manifest.json`.

Replace the fixed `1/sqrt(24)` with normalisation against the operators that actually sound, targeting roughly -12 dBFS peak for one operator at level 1.0 and master 0.8, and add a master output scope reading the existing `main` scope tap so the level is visible. Re-level the four factory presets in the same commit.

Expected: a 1-op patch and a 4-op 6-note chord land within about 6 dB of each other, and no setting of master alone clips. *Test:* extend the offline harness to assert peak dBFS for the matrix {1,2,4 operators} x {1,6 notes held} stays inside a stated window; by ear, A/B against a reference synth at the same master setting. *Note:* this changes the loudness of every existing patch — do it before the preset library exists, not after.

4. The const fast path. New `plugin/ConstHash.h`, `plugin/PluginProcessor.cpp` (`dispatchStateChange` / `scheduleStateDispatch`), `dsp/synth.js` (`ctrl` key map).

I verified the shape to implement: node id = `hashNode('const', props, [])` = `hashString(hashString(-2128831035, "const"), key) & 0x7FFFFFFF`, where `hashString` mixes with `r = imul(r ^ c, 16777619)` over `charCodeAt(0..length)` inclusive — the final iteration contributes `0`, because `charCodeAt(length)` is `NaN | 0`.

A value change is exactly `[[3, hash, "value", v], [5]]`. Build a static param-id → `ctrl()`-key table (it is already implicit in `toSynthState`), and in `dispatchStateChange` take the fast path when every dirty key is in that table and no structural flag (`wave`, `wtid`, `warpmode*`, `level` crossing zero, `mod*_dst`, `fx_chain`, module add/remove) changed. Fall back to the full rebuild otherwise.

Expected: dragging any continuous knob produces no QuickJS work and no audible zipper. *Test:* a Node test that asserts the C++ hash (exercised through a tiny harness, or reimplemented identically in JS) matches `@elemaudio/core`'s `createNode` hash for at least 20 real keys from the shipping graph — `master`, `glide`, `filtA:fc`, `v0:op0:wtpos`, `fx:dly:mix`, and so on; plus a log assertion that a 5-second knob drag on the heaviest patch logs zero reconciles and the audio keeps tracking.

5. Listen-test and triage the arp, sampler and glide. `plugin/ArpEngine.h`, `dsp/synth.js` (sampler branch, glide), `spike/elementary/todo.md`.

No code first — an evidence pass. Work the owner's own unverified list: arp steadiness at 8-20 Hz, each of the 9 directions, dice at 0% and 100%, Auto-Regen on the bar, chord and arp layers, tight-release audibility, per-step slide, the 6 seq mod targets (remembering they read voice 0 only), sampler load/root/loop/start/end/reverse, Rec from Input, and glide feel at several settings. Record what is wrong with a reproduction for each.

Expected: a triaged defect list in `todo.md` replacing the current blanket "unverified by ear" line. *Test:* every finding gets a regression test in `engine.test.mjs` where it is expressible offline, and a written reproduction where it is not. This is the gate for P1 — nothing in band 3 should start while a subsystem this large has never been heard.

One open question that needs a decision before P4

User wavetable ids are **positional** (`faze::uwt::kUserBaseId + index`), so a preset referencing a resynthesised table breaks on any other machine, and even on the same machine if files are added or removed. This must be settled — name plus content hash, or embed the table in the preset — before the first public preset uses one. It is cheap now and expensive once presets are in the wild.

Sources

Competitor facts in this audit come from these pages; everything about FazeSpike itself was read or measured in the repository.

- Serum 2 — Specs, Price & Dubspot Score (2026) — engine modes, price, preset and wavetable counts

- Serum 2 vs Vital: do you need to pay for pro-grade synthesis (MusicTech) — Serum 2's two warp engines and 32 warp options; Vital's 15 spectral and 6 wave morph modes
- Vital Audio — free tier, wavetable converters, text-to-wavetable
- Arturia Pigments 7 review (MusicTech) — engines, generative sequencing, pricing
- Pigments 7 (Plugin Boutique) — v7 content counts
- Phase Plant (Dubspot) — price, Snapins, audio-rate modulation between generators
- Elementary v0.11.0 & new pricing model — checked for a licensing blocker; superseded for this project, since `@elemaudio/core@4.0.1` and the vendored runtime are both MIT in the installed tree